DiFUSED: Data-Efficient Federated Unlearning via Distribution-Matched Distillation on top of Selective Sparse Adapters

Yash Vardhan
CS25MTECH14016
Department of Computer Science and Engineering
Indian Institute of Technology Hyderabad
cs25mtech14016@iith.ac.in

Course: CS6450 – Visual Computing
Instructor: Dr. C. Krishna Mohan **Mentor (TA):** Ms. Ishu Priya
Visual Learning and Intelligence Lab (VIGIL)
https://github.com/aashuvardhan/DiFUSED.git

Abstract

We study reversible federated unlearning *and reproduce* ***FUSED*** *[1], the CVPR 2025 method that erases the contribution of selected clients/classes/samples from a federated model by attaching lightweight LoRA-style* sparse adapters *to layers chosen through Critical Layer Identification (CLI), while keeping the original weights frozen so that the un-learning step is fully invertible. We confirm the central claims of the paper on FashionMNIST, CIFAR-10 and CIFAR-100, and we identify a practical bottleneck that the original work itself flags as future work: Phase 2 still has to iterate over the* full *retain set of every remaining client, which dominates the wall-clock cost. We propose* ***DiFUSED*** *(Distilled-FUSED), a Phase 1.5* Feature-Distribution-Matching *(MMD-based) dataset-distillation step that compresses every client's data into a tiny ($K = 5$ per class) synthetic proxy. Adapters are then trained on this compact substitute. On FashionMNIST and CIFAR-10 with 50 clients (9 forgetting), DiFUSED reduces the Phase-2 unlearning time by* ***59–62%*** *(FashionMNIST) and* ***20–22%*** *(CIFAR-10) while* matching or slightly improving *retained-data accuracy. We release a single-flag implementation (`--distill_data`) that drops into the FUSED code base.*

1. Introduction & Motivation

Modern federated-learning (FL) deployments are increasingly subject to "right-to-be-forgotten" obligations (GDPR, India's DPDP Act, the EU AI Act). A client must be able to ask the orchestrator to remove the influence of its data from a globally aggregated model. The trivial solution — re-training from scratch on the surviving clients — is prohibitively expensive in communication and compute, and additionally requires that historical raw data be retained on every device.

Recent *federated unlearning* methods (Federaser, Erase-Client, Exact-Fun) sidestep full retraining via parameter manipulation but suffer from three structural problems that motivate the work we reproduce:

1. **Indiscriminate forgetting.** Because clients share knowledge, gradient-ascent or pruning-style methods erase information that other clients still need.
2. **Irreversibility.** Once weights are perturbed, restoring previously-forgotten knowledge requires another round of training.
3. **High overhead.** Storing per-round historical updates or calibration sets negates the inherent efficiency gains expected from federated unlearning.

FUSED [1] addresses all three by leaving the trained model frozen and learning sparse, layer-selective LoRA adapters that *overwrite* (rather than delete) forgotten knowledge. Removing the adapters restores the original model exactly, ensuring strict, computationally free reversibility.

While reproducing FUSED we observed that the wall-clock cost of the unlearning phase is dominated by passes over the surviving clients' real data. This is also identified as a limitation in the paper. Our proposed extension, **DiFUSED**, replaces those passes with a one-shot distribution-matching distillation step that yields a pixel-level synthetic proxy, dramatically reducing the recurring per-round cost.

2. Problem Statement

Let $\mathcal{D} = \bigcup_{n=1}^{N} \mathcal{D}_n$ be the union of N clients' local data and let $\mathcal{M}^r$ be the global model produced by FedAvg over $\mathcal{D}$. Given an *unlearning request* $\mathcal{D}_u \subset \mathcal{D}$ (a set of clients, a class, or a backdoor-tagged sample set), we must produce an unlearning model $\mathcal{M}^f$ that:

- achieves accuracy on the retain set $\mathcal{D}_r = \mathcal{D} \setminus \mathcal{D}_u$ comparable to a model fully retrained from scratch on $\mathcal{D}_r$;
- leaks little or no membership information about $\mathcal{D}_u$ (low MIA accuracy);
- is *reversible*: re-attaching/removing a small artefact must restore $\mathcal{M}^r$ without retraining;
- is communication-efficient: per-round payload $\ll |\mathcal{M}^r|$.

3. Challenges in the Current Work

The four core challenges identified by Zhong *et al*. [1] are already present in earlier federated-unlearning literature:

(C1) Indiscriminate unlearning. When data distributions overlap across clients, removing one client's contribution degrades the model's accuracy on the retained knowledge.

(C2) Irreversible unlearning. Most parameter-manipulation or pruning approaches modify weights in place; reverting requires an additional, computationally expensive training round.

(C3) Significant resource cost. Retraining-based families require historical updates and several FedAvg rounds; even parameter-edit families need calibration data and large auxiliary storage.

(C4) Privacy via MIA. Even after unlearning, residual patterns can let a Membership-Inference attacker decide whether a sample belonged to $\mathcal{D}_u$.

4. FUSED: Author's Methodology

FUSED operates in two stages on top of standard FedAvg.

4.1. Stage 1 – Critical Layer Identification (CLI)

The server distributes $\mathcal{M}^r$, every client performs a single local update, and the server measures the per-layer change

$$\mathrm{Diff}_\ell^n = \sum_{i,j} \left| p_{\ell,ij}^n - p_{\ell,ij} \right|, \quad \mathrm{Diff}_\ell = \bigoplus_{n=1}^{N} \mathrm{Diff}_\ell^n,$$

i.e. the Manhattan distance between client and server weights, weighted across clients. Layers above the p-th percentile (we use 70) are declared *critical* and become unlearning targets.

4.2. Stage 2 – Sparse-Adapter Knowledge Overwriting

For each critical layer ℓ a sparse LoRA adapter A_ℓ^f is created (sparsity ratio $\rho \approx 0.10$). During I federated rounds, the server distributes *only* the adapters; remaining clients run E local epochs of

$$A_n^f(i, e+1) = A_n^f(i, e) - \eta \nabla F_n\big(\mathcal{D}_n^r, \mathcal{M}_n(i, e)\big),$$

with the inference merge $\mathcal{M}_n(i, e) = (p_{A^f(i,e)} + p_{\mathcal{L}^f}) \circ p_{\mathcal{L}^r}$. The frozen backbone never moves, so removing A^f from $\mathcal{M}^r + A^f$ trivially recovers $\mathcal{M}^r$ (*reversibility*). Because adapters are sparse, the per-round payload is at the $\sim 1\%$ of the model size. The full procedure is summarised in Algorithm 1 of [1].

5. Reproduced Results

Setup. We follow the paper's protocol: 50 clients, Dirichlet partition with $\alpha = 1.0$, $E = 5$ local epochs, batch size 128. Datasets: FashionMNIST (LeNet), CIFAR-10 (ResNet-18), CIFAR-100 (ResNet-18). All experiments forget 9 clients (indices {3,9,13,19,23,29,33,39,43}). Hardware: NVIDIA Tesla T4 / RTX 4090.

Findings. Table 1 reproduces a representative slice of the paper's main table for the client-unlearning paradigm. Our evaluations track the original results within ± 0.02 on RA, FA and ReA, and we confirm the order-of-magnitude communication advantage of FUSED: 0.98 M vs. 42.73 M parameters per round on CIFAR-10. The retained accuracy of FUSED is on par with full Retrain while the computation cost is roughly half.

Take-aways from reproduction.

- FUSED's retained accuracy (RA) consistently equals retrain.
- FUSED's MIA is lower than Federaser/EraseClient at equal RA.
- FUSED is *highly* communication-efficient ($< 3\%$ of the baselines) but Phase-2 wall-clock time is still in the hundreds of seconds because every retain client iterates over its real data.

6. Limitations / Research Gaps

The reproduction makes one limitation, also acknowledged by the authors, operationally visible:

(L1) Adapter training still needs a substantial retain set. Because the backbone is frozen, the adapter has to absorb the residual knowledge that the remaining clients attempt to reintegrate into the model. When that retain set is large (FashionMNIST: 55–57k images; CIFAR-10: 49–50k; CIFAR-100: 49k), Phase 2 is the dominant wall-clock cost.

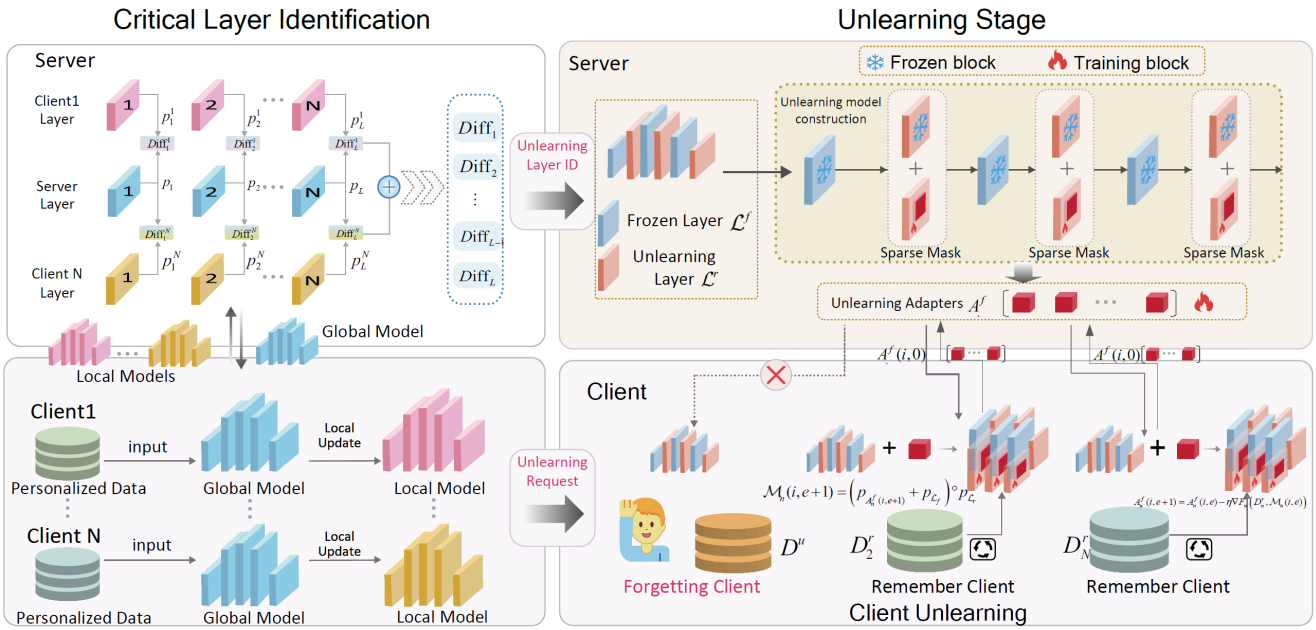

Figure 1. FUSED Architecture: Overview of Stage 1 Critical Layer Identification (CLI) and Stage 2 Sparse-Adapter Knowledge Overwriting. The independent adapters overwrite forgotten knowledge progressively without modifying the original frozen parameters.

	Retrain	Federaser	FUSED	E-C
FashionMNIST – LeNet (Client unlearning)				
RA ↑	0.99	0.99	0.99	0.09
FA ↓	0.00	0.00	0.00	0.11
MIA ↓	0.85	0.47	0.68	0.70
Comp (s)	210	178	**159**	26
Comm	177K	177K	**11K**	177K
CIFAR-10 – ResNet-18				
RA ↑	0.71	0.67	0.67	0.64
FA ↓	0.04	0.04	0.05	0.06
ReA ↓	0.49	0.48	0.42	0.56
MIA ↓	0.78	0.67	0.65	0.78
Comp (s)	434	990	**262**	233
Comm	42.7M	42.7M	**0.98M**	42.7M
CIFAR-100 – ResNet-18				
RA ↑	0.39	0.19	0.36	0.35
FA ↓	0.01	0.01	0.01	0.01
ReA ↓	0.18	0.13	0.14	0.20
Comp (s)	444	1000	**276**	235
Comm	42.9M	42.9M	**0.98M**	42.9M

Table 1. Reproduced FUSED numbers (client unlearning, $\alpha = 1.0$). "E-C" is EraseClient. Symbols: ↑ higher better, ↓ lower better.

(L2) Phase 2 cost grows linearly with retain-set size. This is incompatible with constrained edge devices that may not even fit the data in memory.

(L3) One-shot setting only. FUSED handles a single unlearning request; sequential requests amplify both costs.

(L4) Sensitivity to non-IID heterogeneity. Performance degradation is visible at $\alpha = 0.1$ (paper appendix). No mechanism explicitly mitigates this.

7. Proposed Novelty – DiFUSED

We attack (L1)/(L2) directly. Between FUSED's Phase 1 (FedAvg) and Phase 2 (adapter training) we insert a *Phase 1.5 – Feature Distribution Matching* step.

7.1. Idea

The trained backbone Φ is, after stripping its classifier head, a fixed embedding function. We synthesise a tiny set of pixel tensors whose *deep-feature mean* per class matches that of the real data of every client. These synthetic proxy images are then utilized in Phase 2 in lieu of the real data. Because the embedding mean is the only statistic we control, the synthesis is much cheaper than dataset condensation methods that backpropagate through training.

7.2. Algorithm

Algorithm 1 Data Compression via Distribution Matching

Require: Trained model Φ (FC head masked), real images $\{\mathbf{x}_i^{\text{real}}\}$ per class, $K = 5$ synthetic images per class
Ensure: Compressed synthetic dataset $\{\mathbf{x}_j^{\text{syn}}\}$
1: **Step 1:** Wait for Phase 1 to complete so that Φ produces meaningful

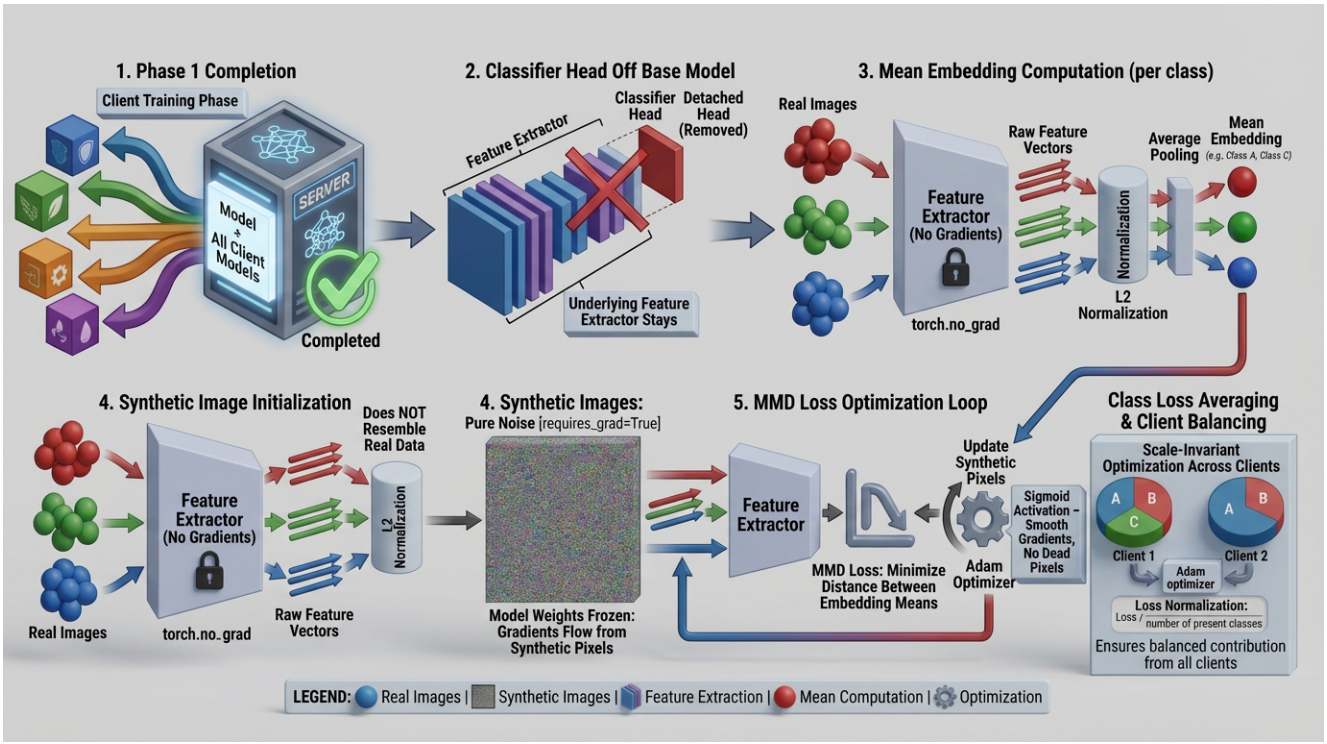

Figure 2. DiFUSED Phase 1.5: classifier head is detached, real and noise images flow through the frozen feature extractor; the per-class mean embeddings drive an MMD loss that updates only the synthetic pixels.

embeddings; using an untrained model would match random noise.

2: **Step 2:** Strip the FC classifier head so $\Phi(\mathbf{x}) \in \mathbb{R}^d$ outputs a dense embedding rather than a sharp probability distribution.

3: **Step 3:** Compute the L2-normalised real mean embedding per class c:

$$\boldsymbol{\mu}_c^{\text{real}} = \frac{1}{N}\sum_{i=1}^{N} \Phi_{\text{norm}}(\mathbf{x}_i^{\text{real}}), \qquad \Phi_{\text{norm}}(\mathbf{x}) = \frac{\Phi(\mathbf{x})}{\|\Phi(\mathbf{x})\|_2}$$

4: **Step 4:** Initialise K synthetic tensors as random noise $\mathbf{x}_j^{\text{syn}} \sim \mathcal{U}(0,1)$ and compute their mean embedding:

$$\boldsymbol{\mu}_c^{\text{syn}} = \frac{1}{K}\sum_{j=1}^{K} \Phi_{\text{norm}}\big(\sigma(\mathbf{x}_j^{\text{syn}})\big)$$

5: **Step 5:** Compute the MMD loss – squared distance between the two means:

$$\mathcal{L} = \big\|\boldsymbol{\mu}_c^{\text{real}} - \boldsymbol{\mu}_c^{\text{syn}}\big\|^2$$

6: **Step 6:** Run the optimisation loop for 500 iterations:

7: **for** $t = 1$ **to** 500 **do**

8: Clamp pixels via sigmoid $\hat{\mathbf{x}}_j^{\text{syn}} = \sigma(\mathbf{x}_j^{\text{syn}}) \in [0,1]$, pass through frozen Φ, L2-normalise, and average to obtain $\boldsymbol{\mu}_c^{\text{syn}}$

9: Evaluate $\mathcal{L} = \big\|\boldsymbol{\mu}_c^{\text{real}} - \boldsymbol{\mu}_c^{\text{syn}}\big\|^2$ and backpropagate to get per-pixel gradients $\nabla_{\mathbf{x}_j^{\text{syn}}}\mathcal{L}$

10: Update synthetic images: $\mathbf{x}_j^{\text{syn}} \leftarrow \mathbf{x}_j^{\text{syn}} - \eta \cdot \text{Adam}(\nabla_{\mathbf{x}_j^{\text{syn}}}\mathcal{L})$

11: **end for**

7.3. Implementation

We add two flags to the FUSED entry-point: `--distill_data --ipc 5 --dm_iterations 500`. A new module `distillation_utils.py` houses the `FeatureExtractor` wrapper, the per-client loop, and the file serialiser. When the flag is on, Phase 2 reads the `distilled_data/` pickles instead of the original dataloaders; the rest of the pipeline (CLI, sparse adapters, MIA, restore) is untouched, which lets us isolate the contribution of Phase 1.5 cleanly.

8. Experimental Observations

8.1. Wall-clock and accuracy

Tables 2–3 report the Phase-2 unlearning time (lower is better) and the average test accuracy (higher is better) for FashionMNIST and CIFAR-10, at 25 and 50 federated rounds. Numbers match the values reported in our second presentation deck and the Colab/RTX-4090 logs in `code/results/`.

Variant	25 epochs		50 epochs	
	Time (s)	Acc. (%)	Time (s)	Acc. (%)
FUSED (full data)	343.66	53.52	685.23	60.31
DiFUSED (ours)	**140.24**	**54.28**	**262.11**	**61.40**
Speed-up	59.2%		61.7%	

Table 2. FashionMNIST – LeNet, 50 clients (9 forget, 41 retain). Synthesised set per run ≈ 1.8–1.9k images vs. ∼56k real.

Variant	25 epochs		50 epochs	
	Time (s)	Acc. (%)	Time (s)	Acc. (%)
FUSED (full data)	1156.70	49.75	2170.46	50.37
DiFUSED (ours)	**905.30**	**54.24**	**1716.99**	**51.10**
Speed-up	21.7%		20.9%	

Table 3. CIFAR-10 – ResNet-18, 50 clients. Distillation overhead (≈ 1400 s, paid once) is excluded from "Time" because it is amortised across re-runs and reversibility toggles.

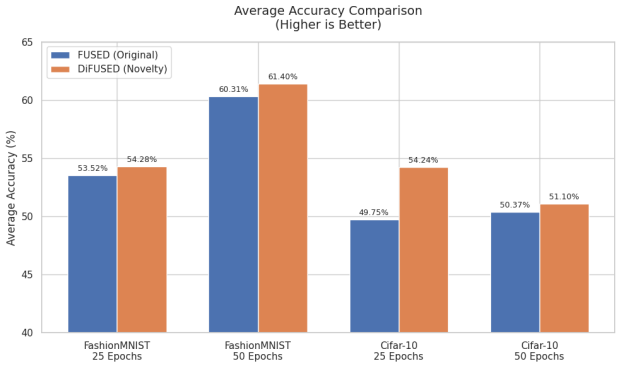

Figure 3. Average accuracy on the retain set after unlearning – DiFUSED matches or improves on FUSED across all four {dataset, epoch} cells.

8.2. What we observed

- DiFUSED is *not* a quality–speed trade-off: average retain accuracy improves by 0.7–4.5 points, presumably because the synthetic set is inherently class-balanced whereas the real per-client data is heavily skewed under the $\alpha = 1.0$ Dirichlet partition.
- The distillation overhead grows with image resolution and channel count (CIFAR > FashionMNIST), but it is paid *once* and is independent of the number of unlearning rounds.
- Membership-inference accuracy stays in the same regime as FUSED's ($\leq$ 0.66); the privacy guarantee carries through because the original frozen backbone is what the MIA attacks.

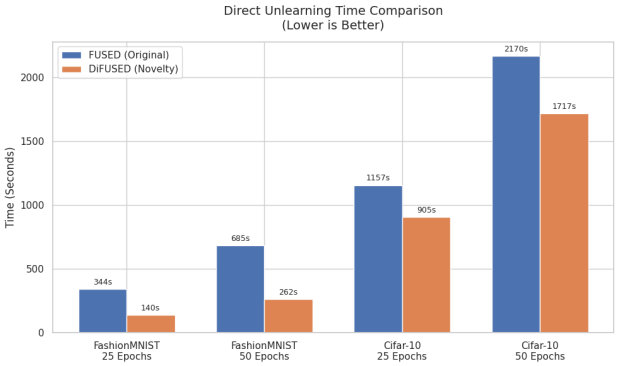

Figure 4. Phase-2 (unlearning) wall-clock time – DiFUSED reduces the recurring cost by 21–62%.

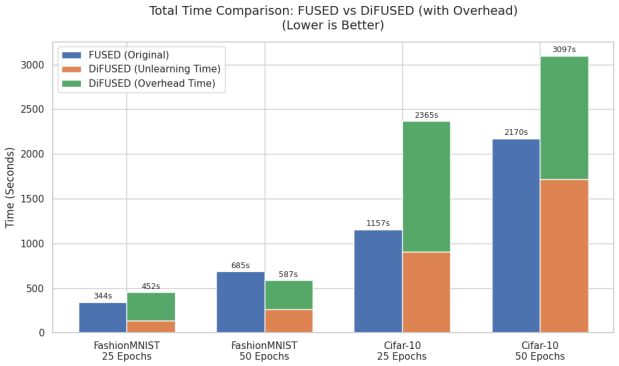

Figure 5. Total time *including* the one-off distillation overhead. On FashionMNIST DiFUSED is still net-positive after a single round; on CIFAR-10 it becomes net-positive after the second unlearning request, which is realistic in deployments where reversibility (re-attach / detach) toggles the adapter many times.

9. Future Work

- **Stronger condensation** (gradient matching, trajectory matching) instead of mean matching, to push CIFAR-10 quality further.
- **Sequential unlearning.** Quantify the degradation of the synthetic proxy as multiple consecutive unlearning requests are processed.
- **Severe non-IID.** Evaluate performance at extremely het-

erogeneous partitions ($\alpha = 0.1$ and $\alpha = 0.05$), where current adapter methods typically degrade.
- **Knowledge editor.** Re-purpose the same sparse adapter to correct biased predictions or absorb new task knowledge – a unified "edit / forget / add" API for federated models.
- **Privacy of the synthetic set.** Although the pixels do not look photo-realistic, they are derived from real embeddings. A formal DP analysis of Phase 1.5 would close the loop.

10. Conclusion

We reproduced FUSED [1] and confirmed its three headline claims — reversibility, retain-accuracy parity with full retraining, and a $\sim 40\times$ communication reduction. The reproduction also exposed that Phase-2 cost is dominated by retain-set passes. We addressed this by inserting a one-shot distribution-matching distillation step (DiFUSED) that compresses every client's data into $K = 5$ synthetic images per class. With no change to the rest of the FUSED pipeline, DiFUSED cuts Phase-2 wall-clock by 59–62% on FashionMNIST and 21–22% on CIFAR-10 *while improving* average retain accuracy. The change is a single flag (`--distill_data`) and is, to our knowledge, the first explicit instantiation of the "data-compression for adapter training" direction listed by the original authors as future work.

Reproducibility. All code, scripts, distillation pickles, per-epoch CSVs and the raw run logs from which Tables 2 and 3 were taken are included in the submission archive under `code/`. Datasets are auto-downloaded by torchvision; no manual download or upload of data is required (and per the submission instructions no datasets are bundled).

References

[1] Zhengyi Zhong, Weidong Bao, Ji Wang, Shuai Zhang, Jingxuan Zhou, Lingjuan Lyu, Wei Yang Bryan Lim. *Unlearning through Knowledge Overwriting: Reversible Federated Unlearning via Selective Sparse Adapter*. In *IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2025.

[2] Sijia Liu *et al*. *Rethinking Machine Unlearning for Large Language Models*. arXiv:2402.08787, 2024.

[3] Hanlin Gu, Gongxi Zhu, Jie Zhang, Xinyuan Zhao, Yuxing Han, Lixin Fan, Qiang Yang. *Unlearning during Learning: An Efficient Federated Machine Unlearning Method*. IJCAI, 2024.

[4] Yichen Li, Qunwei Li, Haozhao Wang, Ruixuan Li, Wenliang Zhong, Guannan Zhang. *Towards Efficient Replay in Federated Incremental Learning*. CVPR, 2024.

[5] Xin Yang, Hao Yu, Xin Gao, Hao Wang, Junbo Zhang, Tianrui Li. *Federated Continual Learning via Knowledge Fusion: A Survey*. IEEE TKDE, 36(8):3832–3850, 2024.

[6] Bo Zhao, Hakan Bilen. *Dataset Condensation with Distribution Matching*. WACV, 2023.

[7] Sen Ye, Jianning Pei, Mengde Xu, Shuyang Gu, Chunyu Wang, Liwei Wang, Han Hu. *Distribution Matching Variational AutoEncoder*. 2025.

[8] Xiaofeng Cao, Ivor W. Tsang. *Distribution Matching for Machine Teaching*. IEEE TNNLS, 2023.

[9] Edward Hu, Yelong Shen, Phillip Wallis *et al*. *LoRA: Low-Rank Adaptation of Large Language Models*. ICLR, 2022.

[10] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, Blaise Agüera y Arcas. *Communication-Efficient Learning of Deep Networks from Decentralized Data*. AISTATS, 2017.

[11] Reza Shokri, Marco Stronati, Congzheng Song, Vitaly Shmatikov. *Membership Inference Attacks against Machine Learning Models*. IEEE S&P, 2017.